

Personal Technical Reference & Study Notes: Transformer Block & Attention Mechanisms

Comprehensive Synthesis from *Attention is All You Need* (Vaswani et al.) & DeepLearning.AI Coursework

Generated for Knowledge Collection & Code Reference — Last Updated: September 2026

1 Macro Architecture & Pipeline Overview

The Transformer architecture, introduced by Vaswani et al. (2017), completely dispenses with recurrent neural networks (RNNs) and convolutional networks (CNNs), relying entirely on self-attention mechanisms to establish global dependencies between input and output tokens.

In traditional sequence modeling (e.g., LSTMs and GRUs), computation is factored sequentially along symbol positions, computing hidden state h_t as a function of h_{t-1} and input x_t . This sequential dependency creates a fundamental bottleneck, precluding parallelization across sequence positions during training. The Transformer resolves this bottleneck by processing all context tokens simultaneously across parallel tensor tracks.

Transformer Architecture Overview

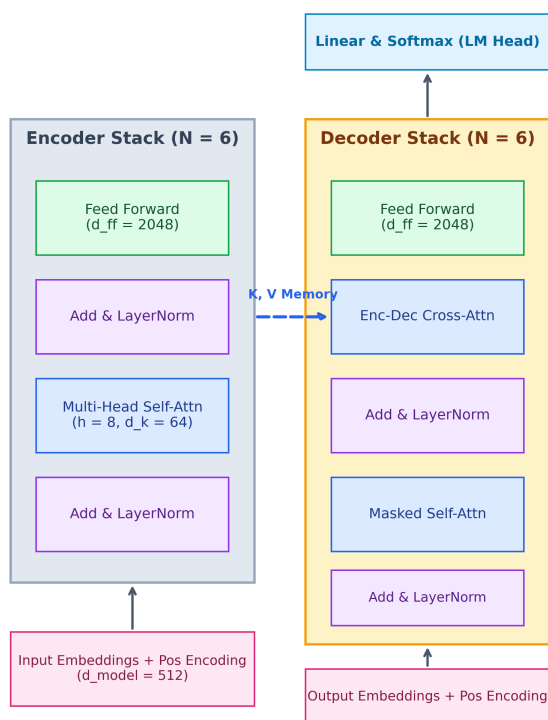


Figure 1: **Macro Architecture of the Transformer Encoder-Decoder Stacks.** Tokens are mapped to continuous embeddings ($d_{\text{model}} = 512$), combined with sinusoidal positional encodings, and passed through $N = 6$ stacked encoder and decoder blocks with residual connections and layer normalization.

1.1 Encoder and Decoder Stacks

- **Encoder Stack ($N = 6$):** Composed of $N = 6$ identical layers. Each layer contains two sub-layers: (1) a multi-head self-attention mechanism, and (2) a position-wise fully connected feed-forward network. Around each sub-layer, a residual connection is employed followed by layer normalization:

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x)) \quad (1)$$

To facilitate residual connections, all sub-layers and embedding layers produce output vectors of dimension $d_{\text{model}} = 512$.

- **Decoder Stack ($N = 6$):** Also composed of $N = 6$ identical layers. In addition to the two sub-layers present in each encoder block, the decoder inserts a third sub-layer that performs **Encoder-Decoder Cross-Attention** over the output memory stack of the encoder. The self-attention sub-layer in the decoder is modified with causal masking (setting illegal connections to $-\infty$) to prevent positions from attending to subsequent future tokens, preserving the auto-regressive property.
- **Language Modeling (LM) Head:** The final vector for the prompt tokens flows to a linear layer followed by a pre-softmax transformation sharing weights with the input embeddings (scaled by $\sqrt{d_{\text{model}}}$), outputting predicted next-token probabilities across the vocabulary.

2 Scaled Dot-Product Attention & Variance Scaling

An attention function maps a set of query vectors (Q), key vectors (K), and value vectors (V) to an output vector. The output is computed as a weighted sum of the value vectors, where the weight assigned to each value is computed by a compatibility function of the query with its corresponding key.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (2)$$

2.1 Mathematical Proof: Scaling Factor $\frac{1}{\sqrt{d_k}}$

For small key dimensions d_k , additive attention and dot-product attention perform similarly. However, for large values of d_k , unscaled dot products grow large in magnitude, pushing the softmax function into regions with extremely small gradients.

Proof of Dot Product Variance Scaling. Assume the components of query $q \in \mathbb{R}^{d_k}$ and key $k \in \mathbb{R}^{d_k}$ are independent random variables with mean $E[q_i] = E[k_i] = 0$ and variance $\text{Var}(q_i) = \text{Var}(k_i) = 1$. The dot product is:

$$q \cdot k = \sum_{i=1}^{d_k} q_i k_i \quad (3)$$

The expected value of the product $q_i k_i$ is $E[q_i k_i] = E[q_i]E[k_i] = 0$. Since q_i and k_i are independent:

$$\text{Var}(q_i k_i) = E[q_i^2 k_i^2] - (E[q_i k_i])^2 = E[q_i^2]E[k_i^2] - 0 = \text{Var}(q_i)\text{Var}(k_i) = 1 \cdot 1 = 1 \quad (4)$$

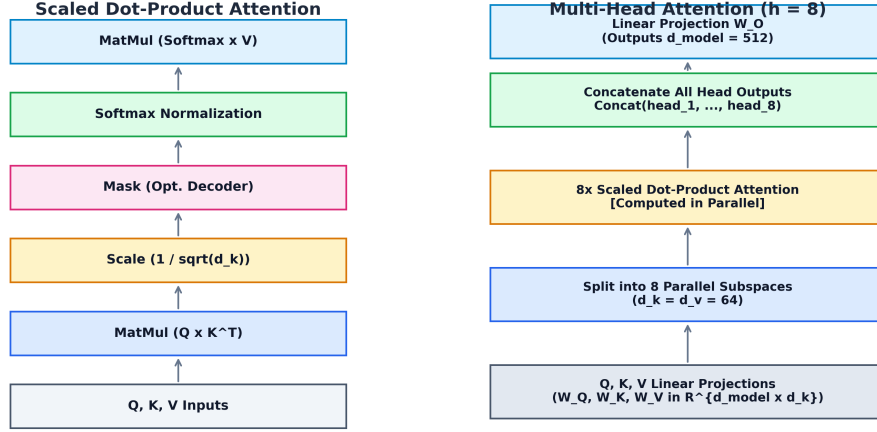


Figure 2: **Execution Flowcharts.** Left: Scaled Dot-Product Attention pipeline. Right: Multi-Head Attention linear projection and parallel head execution ($h = 8$).

Summing across all d_k dimensions:

$$\text{Var}(q \cdot k) = \text{Var}\left(\sum_{i=1}^{d_k} q_i k_i\right) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k \cdot 1 = d_k \quad (5)$$

Thus, the variance of the dot product grows linearly with dimension d_k (standard deviation $\sqrt{d_k}$). Dividing by $\sqrt{d_k}$ scales the variance back to 1, ensuring stable softmax behavior and preventing vanishing gradients. $\square$

3 Multi-Head Attention

Rather than performing a single attention function with d_{model} -dimensional vectors, Multi-Head Attention projects queries, keys, and values h times with learned linear projections to d_k , d_k , and d_v dimensions, respectively.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (6)$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (7)$$

Where parameter matrices are:

$$W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}, \quad W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}} \quad (8)$$

In the standard base configuration: $h = 8$ parallel heads, $d_{\text{model}} = 512$, and $d_k = d_v = d_{\text{model}}/h = 64$. Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions (e.g., tracking syntactic agreements on one head while capturing semantic long-range dependencies on another).

4 Self-Attention Intuition & Coreference Resolution

As detailed in DeepLearning.AI coursework (Jay Alammar & Maarten Grootendorst), self-attention consists of two major stages: **relevance scoring** and **combining information**.

1. **Relevance Scoring:** Computes dot-product compatibility scores between the current token query and all previous token keys, producing a normalized softmax score distribution across positions.
2. **Combining Information:** Multiplies the attention scores by the value vectors (V) and sums them up, effectively "baking" context from relevant prior tokens into the current token representation vector.

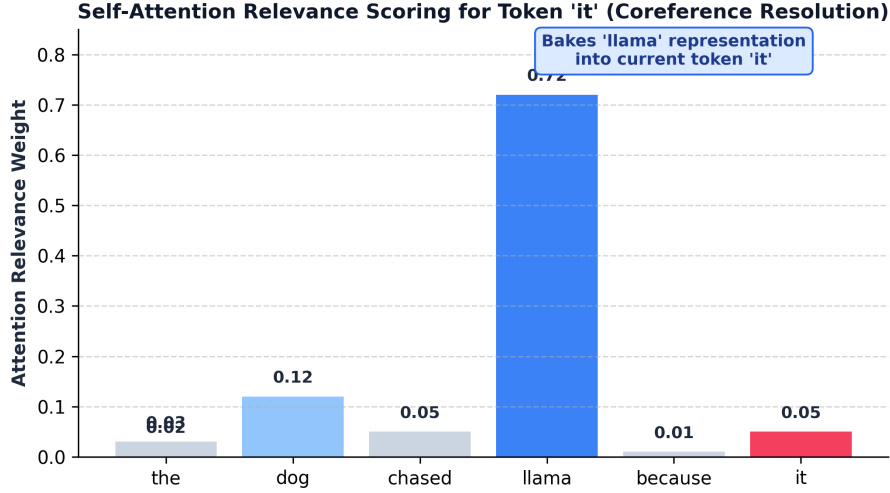


Figure 3: **Coreference Resolution via Self-Attention.** When evaluating the prompt *"the dog chased the llama because it"*, self-attention assigns a high relevance weight (0.72) between *"it"* and *"llama"*, baking the llama representation vector directly into *"it"*.

5 Position-wise Feed-Forward Networks & Sinusoidal Encodings

5.1 Feed-Forward Networks (FFN)

Each block in the encoder and decoder contains a fully connected feed-forward network applied to each position separately and identically:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (9)$$

The dimensionality of input and output is $d_{\text{model}} = 512$, while the inner-layer dimensionality expands to $d_{\text{ff}} = 2048$. The network acts as a high-capacity feature expansion and storage mechanism for domain statistical patterns.

5.2 Sinusoidal Positional Encoding

Because the model contains no recurrence or convolutions, positional encodings are added to the input embeddings to supply sequence order information:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (10)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (11)$$

where pos is position and i is dimension. Wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$.

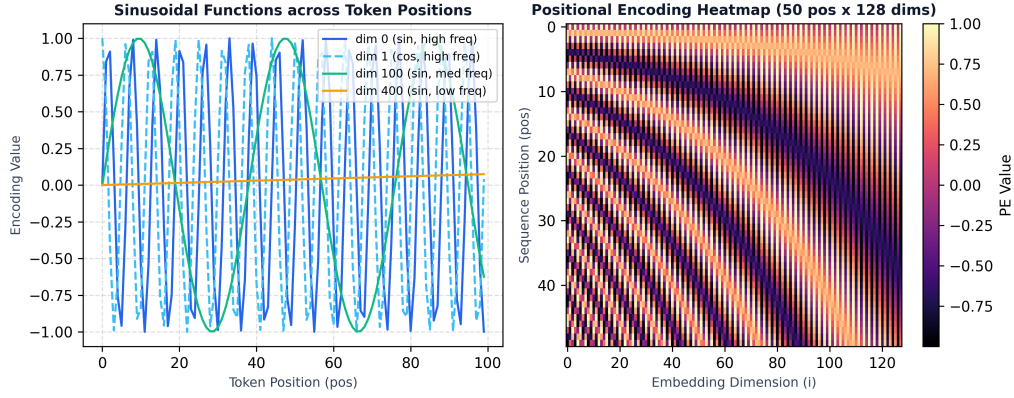


Figure 4: **Sinusoidal Positional Encodings.** Left: Sine and cosine frequency curves across positions. Right: Heatmap visualization displaying orthogonal geometric patterns across 128 embedding dimensions.

6 Theoretical Computational Complexity Analysis

Table 1 highlights why self-attention replaced recurrent architectures for long-sequence modeling.

Table 1: Per-Layer Computational Complexity, Sequential Operations, and Maximum Path Lengths.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent (RNN)	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Restricted Self-Attention	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

When sequence length $n < d$ (standard in NLP where $n \leq 512$ and $d = 512$), $O(n^2 \cdot d) < O(n \cdot d^2)$. Furthermore, maximum path length between any two tokens drops to $O(1)$, greatly facilitating gradient propagation for long-range dependencies.

7 Training Optimization & Hyperparameters

- **Optimizer:** Adam optimizer with $\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 10^{-9}$.
- **Learning Rate Schedule:** Varied according to warmup steps ($warmup_steps = 4000$):

$$lr_{rate} = d_{model}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}) \quad (12)$$

- **Regularization:** Residual Dropout $P_{drop} = 0.1$ applied to sub-layer outputs and embedding sums, along with Label Smoothing $\epsilon_{ls} = 0.1$.

8 PyTorch Code Implementations

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import math
5
6 class ScaledDotProductAttention(nn.Module):
7     def __init__(self):
8         super().__init__()
9
10    def forward(self, Q, K, V, mask=None):
11        d_k = Q.size(-1)
12        # 1. Relevance Scoring:  $Q * K^T / \sqrt{d_k}$ 
13        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
14
15        if mask is not None:
16            scores = scores.masked_fill(mask == 0, -1e9)
17
18        attn_weights = F.softmax(scores, dim=-1)
19        # 2. Combining Information
20        output = torch.matmul(attn_weights, V)
21        return output, attn_weights
22
23
24 class MultiHeadAttention(nn.Module):
25     def __init__(self, d_model=512, h=8):
26         super().__init__()
27         assert d_model % h == 0, "d_model must be divisible by h"
28         self.d_model = d_model
29         self.h = h
30         self.d_k = d_model // h # 64
31
32         self.W_q = nn.Linear(d_model, d_model)
33         self.W_k = nn.Linear(d_model, d_model)
34         self.W_v = nn.Linear(d_model, d_model)
35         self.W_o = nn.Linear(d_model, d_model)
36
37         self.attention = ScaledDotProductAttention()
38
39     def forward(self, Q, K, V, mask=None):
40         batch_size = Q.size(0)
41
42         # Split into h heads: (batch, h, seq_len, d_k)
43         q_proj = self.W_q(Q).view(batch_size, -1, self.h, self.d_k).transpose(1, 2)
44         k_proj = self.W_k(K).view(batch_size, -1, self.h, self.d_k).transpose(1, 2)
45         v_proj = self.W_v(V).view(batch_size, -1, self.h, self.d_k).transpose(1, 2)
46
47         out, _ = self.attention(q_proj, k_proj, v_proj, mask=mask)
48
49         # Concatenate and project back to d_model
50         out = out.transpose(1, 2).contiguous().view(batch_size, -1, self.d_model)
51         return self.W_o(out)
52
53

```

```

54 class PositionwiseFeedForward(nn.Module):
55     def __init__(self, d_model=512, d_ff=2048):
56         super().__init__()
57         self.w_1 = nn.Linear(d_model, d_ff)
58         self.w_2 = nn.Linear(d_ff, d_model)
59
60     def forward(self, x):
61         return self.w_2(F.relu(self.w_1(x)))
62
63
64 class PositionalEncoding(nn.Module):
65     def __init__(self, d_model=512, max_len=5000):
66         super().__init__()
67         pe = torch.zeros(max_len, d_model)
68         position = torch.arange(0, max_len, dtype=torch.float).
69             unsqueeze(1)
70         div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-
71             math.log(10000.0) / d_model))
72
73         pe[:, 0::2] = torch.sin(position * div_term)
74         pe[:, 1::2] = torch.cos(position * div_term)
75         self.register_buffer('pe', pe.unsqueeze(0))
76
77     def forward(self, x):
78         return x + self.pe[:, :x.size(1)]
79
80 class TransformerEncoderBlock(nn.Module):
81     def __init__(self, d_model=512, h=8, d_ff=2048, dropout=0.1):
82         super().__init__()
83         self.attn = MultiHeadAttention(d_model=d_model, h=h)
84         self.ffn = PositionwiseFeedForward(d_model=d_model, d_ff=d_ff)
85         self.norm1 = nn.LayerNorm(d_model)
86         self.norm2 = nn.LayerNorm(d_model)
87         self.dropout = nn.Dropout(dropout)
88
89     def forward(self, x, mask=None):
90         attn_out = self.attn(x, x, x, mask=mask)
91         x = self.norm1(x + self.dropout(attn_out))
92         ffn_out = self.ffn(x)
93         x = self.norm2(x + self.dropout(ffn_out))
94         return x

```

Listing 1: Complete PyTorch Implementation of Transformer Encoder Components